

Dense retrieval uses dense vector for retrieval.
(Semantic retrieval)

Bi-encoders

fast, but - low retrieval quality

Cross-encoders

slow, but very high retrieval quality

Late-interaction

in the middle for speed and quality.

1. Bi-Encoders

Query $\rightarrow$ encoder $\rightarrow$ embedding

normally same

Each document $\rightarrow$ encoder $\rightarrow$ embedding

During retrieval, we can use Approximate Nearest Neighbors (ANN) such as HNSW finding top-K documents.

Example: Sentence-BERT

fine-tune BERT for embedding sentences

How to use Sentence BERT Comparing the similarity of two paragraphs.

Sentence A $\rightarrow$ BERT $\rightarrow$ mean pooling of $\rightarrow$ Vector u
 $\uparrow$
Same model (tied weights)
embeddings

Sentence B $\rightarrow$ BERT $\rightarrow$ mean pooling $\rightarrow$ Vector v

Cosine Similarity

How is it fine-tuned?

u and v need to be here, because for NLI problem, the order of sentence is crucial. $u-v$ along ignores this.

Why is $u-v$ here?

If A and B are very similar, BERT produce similar u and v , then the model learns that if $u-v$ is small, it is entailment. In other words, BERT and the final 2^T for classification should use the fact $u-v$ is small $\Rightarrow$ entailment. So they will learn to push $u \rightarrow v$ if entailment is true.

(The assumption here is that the neural net will choose easy for things to work)

use this long vector to perform Natural Language Inference task. Classify if B is

- entailment of A
 $A \rightarrow B$
if A is true, B is true
- Contradiction of A
 $A \rightarrow \neg B$
if A is true, B is false
- Neutral
 $A \nrightarrow B$
or
 $A \nrightarrow \neg B$

Pros for bi-encoder: fast and efficient. The documents are all embedded and stored in vector form for retrieving

Cons for bi-encoder: the embedding quality is relatively low.

Cross-encoder (early interaction)

Input = $\langle \text{CLS} \rangle + \text{Query} + \langle \text{SEP} \rangle + \text{Document} + \langle \text{SEP} \rangle$

fine-tuned BERT

relevance score between
the query and document

using contextual embedding
of $\langle \text{CLS} \rangle$

$$\text{Score} = w^T \cdot \langle \text{CLS} \rangle$$

How is BERT fine-tuned for cross-encoder?

1. BERT + sigmoid adapter in the end

classification

1 if query matches the document
0 if not

2. BERT + sigmoid adapter in the end

pairwise loss

$$-\log(\sigma(S(q, d^+) - S(q, d^-)))$$

query matched document unmatched

Pros of cross-encoder:

Why is Cross-encoder much more powerful than bi-encoder?

Because in bi-encoder, query and documents are embedded completely independently.

At best, the embedding of a document cares about:

embedding of a document: what this document
is generally about

the best it can do is to be "average" of the queries embeddings

because it does not see query when embedding, so it should be embedded to work for all related queries.

embedding of the document
embeddings of relevant queries

However, Cross-encoder learns the relevance between the document and the specific query.

Cons of cross-encoder:

It is very slow. Need to embed all documents when doing one retrieval.

Late-interaction (e.g. ColBERT)

↓
Contextualized Late
interaction over BERT

In bi-encoder, we use one embedding vector for query and document.

In ColBERT, the embeddings for each token will be kept.

So a document i with 500 words $\leadsto E_i \in \mathbb{R}^{500 \times d}$ e.g. $d = 1024$
embedding matrix

A query j with 10 words $\leadsto E_j \in \mathbb{R}^{10 \times d}$

$$S(q, d) = \text{Sum} \left(\max \left(E_j \cdot E_d^T, \text{dim} = -1 \right) \right)$$

Sum across tokens in query

maximum similarity

for each token in query, compute the similarity with all tokens in the document, and then take the maximum.

Pros

Why this is better than bi-encoder: token level details vs sentence level

Cons

Take a huge storage room